

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 3.1: Why ML for Economists & Pattern Recognition

● Zhigang Feng

2026-07-06 01:24:48

Topic 3.1 Agenda

1. **Why ML for Economists?**
2. **Pattern Recognition** — the motivating illustration

Arc: *T1 Why ML* → *T2 NN Concepts* → *T3 Training + Backprop* → *T4 Specialized + Lab*

Why ML for Economists?

© Zhigang Feng

Redistribution prohibited without permission

Harnessing the Power of Unstructured Data

- Economics has traditionally relied on structured, tabular data. However, a wealth of economic information is locked in unstructured formats: text, images, satellite imagery.
- ML algorithms, particularly deep learning, provide tools to systematically process and quantify this unstructured data into economic indicators.
- **Applications:**
 - Measuring economic sentiment and uncertainty from financial news and social media
 - Analyzing satellite images to track economic activity (nighttime lights $\approx$ GDP proxy, ships in ports $\approx$ trade flows)
 - Using image recognition on product pictures to understand non-standardized product features
- **Course arc:** Lec. 2 T2 framed AI as cognitive capital — the *why*. This lecture delivers the *what* (NN architectures, training algorithms); Lec. 4 delivers the *how* (solving dynamic equilibrium models).

Extracting Insights from Textual Data

- Natural Language Processing (NLP) offers powerful techniques to analyze large volumes of text — going beyond simple word counts.
- **Applications:**
 - Gauging dovish or hawkish stance of central bankers from public communications
 - Identifying key risk factors from corporate annual reports
 - Quantifying the evolution of economic policy uncertainty from news articles
- **For macro:** FOMC minutes, Beige Books, and congressional records contain rich information that traditional econometrics cannot easily exploit.

Enhancing Causal Inference and Forecasting

- **Causal Inference:**

- “Double Machine Learning” allows flexible inclusion of high-dimensional controls, reducing omitted variable bias
- ML identifies *heterogeneous treatment effects*: how does a policy impact vary across subgroups?

- **Forecasting & Nowcasting:**

- ML captures complex, non-linear relationships that time-series models miss
- Regularization (e.g., LASSO) enables “kitchen sink” regression without overfitting
- Nowcasting: real-time economic indicators from credit card transactions, web searches, satellite imagery

Solving Large-Scale Macro and Finance Models

- Cutting-edge models (heterogeneous agents, portfolio choice) feature:
 - High dimensionality and complex non-linear dynamics
 - Traditional grid-based methods hit the **curse of dimensionality**
- ML techniques provide powerful new numerical methods:
 - **Deep learning**: parameterize policy/value functions with neural networks
 - **Reinforcement learning**: solve sequential decision problems via simulation
- **Examples:**
 - Optimal fiscal/monetary policy in heterogeneous agent models
 - Life-cycle portfolio choice with realistic income uncertainty
 - Krusell–Smith (1998) with neural network approximation of the law of motion

ML and the Curse of Dimensionality in Macro

- **CoD #1: Dimension of the state space**

- $k \rightarrow \{k, a, h, \dots\}$; the distribution Γ is infinite-dimensional
- *Traditional*: sparse grids, local approximation — scale poorly
- *ML*: neural networks are **grid-free**; recover equilibrium functions via Monte Carlo sampling

- **CoD #2: Unfolding of uncertainty**

- $\mathbb{E}$ over future contingencies; the space for Γ is unknown
- *Traditional*: discretize shocks, impose Markov structure
- *ML*: focus on the ergodic distribution; direct simulation Monte Carlo

- **Key insight**: ML shifts the computational bottleneck from *grid construction* to *function representation* — a much more scalable paradigm.

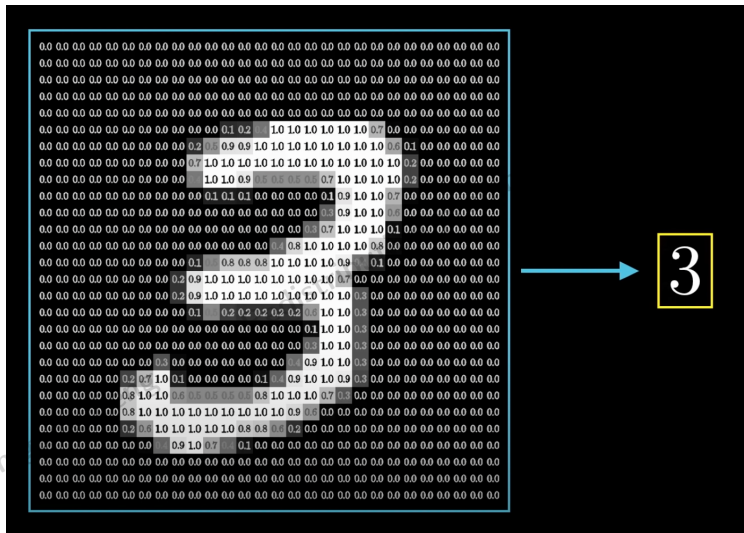
- **Where this gets concrete in the course**: Lec. 4 T1 applies it to the optimal-growth state (k, z) ; Lec. 5 T1 generalizes to MDPs (S, A, P, R, γ) ; Lec. 6 attacks high-dim Aiyagari and Krusell–Smith with the methods you'll learn in T2–T4.

Pattern Recognition as Motivating Illustration

© Zhigang Feng

Redistribution prohibited without permission

Example: Handwritten Digit Recognition



Digit Recognition as a Mathematical Problem

- **Dataset:** $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with images $\mathbf{x}_i \in [0, 1]^{784}$ and labels $y_i \in \{0, \dots, 9\}$
- **Input-Output Mapping:**

$$f_{\theta}: \mathbf{x} \mapsto \hat{\mathbf{p}} \in \Delta^9, \quad \hat{y} = \arg \max_k \hat{p}_k$$

where Δ^9 is the 10-dimensional probability simplex

- **Learning Objective (Empirical Risk + Regularization):**

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \underbrace{\mathcal{L}(f_{\theta}(\mathbf{x}_i), y_i)}_{\text{cross-entropy}} + \lambda \|\theta\|_p$$

- **Search Strategy:** Stochastic gradient descent & variants (Adam, RMSProp)

Constraints and Structure in Handwritten Digits

- **Anatomical constraints:** Muscles, tendons, and joints dictate possible strokes — leading to systematic patterns in how digits are formed.
- **Geometric properties:** Each digit has defining features — ‘0’ has a closed loop, ‘1’ a vertical stroke, ‘8’ two stacked loops.
- **Spatial relationships:** Relative positions, sizes, and orientations of components maintain digit identity even with individual variation.
- **Lesson for economists:** Just as economic theory constrains the space of plausible models, structural constraints on handwriting reduce the effective dimensionality of the recognition problem.
- ML algorithms can **learn** these constraints from data — data augmentation, regularization, and architecture choices all exploit this structure.

From 784 Pixels to a Low-Dimensional Manifold

- **Manifold Hypothesis:** Natural digit images occupy a smooth, d -dimensional manifold $\mathcal{M} \subset \mathbb{R}^{784}$ with $d \ll 784$.
- **Plain-English version:** only a few “writing-style” degrees of freedom matter: slant, stroke thickness, loop closure, and local translation.
- The other 780+ pixel directions are nearly empty: random pixel noise almost never looks like a digit.
- **Macro analogy:** finding a low-dimensional state in a high-dimensional model is the same problem. A good representation preserves the few variables that matter for the next decision.

Encoder View of Digit Recognition

- **Two-stage model:**

$$\mathbf{x} \xrightarrow{\pi: \mathbb{R}^{784} \rightarrow \mathbb{R}^d} \mathbf{z} \xrightarrow{g_\phi: \mathbb{R}^d \rightarrow \Delta^9} \hat{\mathbf{p}}, \quad \hat{y} = \arg \max_k \hat{p}_k$$

- π : *encoder* — flattens the manifold to a latent code $\mathbf{z} \in \mathbb{R}^d$
- g_ϕ : *classifier head* — 1–2 FC layers + softmax $\rightarrow$ class probabilities

- **Choices for π :**

- **Linear:** PCA, SVD, random projections
- **Non-linear:** Auto-encoder, t-SNE, UMAP, first layers of a CNN

- **Economist's translation:** the encoder learns a nonlinear factor model instead of imposing a fixed sufficient statistic. Sufficient statistics for prediction — by learning.

Defining a Deep Neural Network for Digit Recognition

- **Network Architecture:**

- Input Layer: 784 neurons corresponding to pixel intensities
- Hidden Layers: Multiple layers applying nonlinear transformations to learn complex patterns
- Output Layer: 10 neurons, each representing one digit class (0–9)

- **Activation Functions:**

- Nonlinear functions (ReLU, sigmoid) between layers
- Customizable depth and width based on task complexity

- **Economic parallel:** The network structure mirrors the recursive structure in macro models — state variables are transformed layer by layer into decisions, just as $k_t \rightarrow c_t, l_t$ through policy functions.

- **Next (T2):** unpack the building blocks — neurons, layers, activations, and why universal approximation holds.